Name:- Chethukya Sai

Registerno:- 19251112

Course code:- CSA1306

CO2- Assignment

Task 1 - Regular expression design for Input Validation

Introduction:-

A regular expression (RE) is a mathematical notation used to represent patterns in a language. It is commonly used in software applications for Input validation and pattern matching.

The basic operations used in regular expression

1) Union (+):- represents choice b/w two symbols
2) Concatenation (Combines two expression
3) Kleene star (*) represents zero or more repetitions

Regular Expression

(A) Binary string ending with 01

$\Sigma = \{0,1\}$

$RE = [(0+1)*01]$

(B) Binary string containing 11 or 00

$RE = (0+1)^*(11+00)(0+1)^*$

0+1 allows binary prefix

11+00 requires either 11 or 00

(C) String over {a,b} containing atleast one ab

$\alpha = \{ ab, aab, babab \ldots \}$

$RE = (a+b)^* ab (a+b)^*$

$(a+b)^*$ allows sequence of a and b

ab ensures atleast one occurrence of ab

(D) Binary strings having an even number of zeros

$\alpha = \{ \varepsilon, 1, 11, 00, 0101 \ldots \}$

$R.E = 1^*01 \quad (10101^*)1$

Each block contains two zeros

1^* allows any number of ones between zeros

(E) String over {a,b} starting with a and ending with b

$\alpha = ab, aab, abb, aaab, abab$

$R.E = [a(a+b)^*b]$

first symbol must be a

$(a+b)^*$ allows any combination of a and b

The last symbol must be b

APPLICATIONS

Regular expressions are used in

Username and password validation

Searching text patterns

Input validation in programming

Data filtering and text processing

Regular expression provides a simple way to identify and validate input patterns in software applications

TASK 2- Regular Expression Analysis and language

Representation

Given regular expression

$$R = (0+1)^* 101 (0+1)^*$$

1. Language Representation

The given regular expression represents all binary strings containing the substring 101 at least once.

The string may contain any number of zeroes and ones before and after the substring 101

2. Alphabet

$$\Sigma = \{0, 1\}$$

Recognized substring : 101

Accepted String

The following String contain the substring 101

String	Explanation
101	Exact substring
0101	End with 101
1101	contains 101
1010	Starts with 101
00101	contains 101 at the end

Rejected String

The following string do not contain substring 101

String	Reason
000	No occurrence of 101
111	No occurrence of 101
010	Doesnot contain 101

EXPLANATION OF REGULAR EXPRESSION

(A) UNION (+)

Union represents a choice between two symbols

(0+1) It allows either 0 or 1

(B) Kleene star (*)

Kleene star represents zero or more repetitions

$(0+1)^*$

It allows binary string including the empty string ε

c. Concatenation

Concatenation combines expressions to express
In the given expression, the concatenation is done between two 'b' symbols

Algorithm

Declare a string variable
Get the input
Validate the input
Traverse the string in reverse order

Task 3 - Context free Grammar Construction

A context-free grammar (CFG) describes the structure of a language using production rules,

$G = (V, T, P, S)$

V – Set of non-terminals
T – Set of terminals
P – Set of production rules
S – Start symbol

Grammar for $L_1 = \{a^n b^n \mid n \geq 0\}$

Production rule:

$S \rightarrow aSb \mid \varepsilon$

Derivation for aabb

$S \Rightarrow aSb$
$\Rightarrow aaSbb$
$\Rightarrow aabb$

Generated strings: $\varepsilon, ab, aabb, aaabbb \ldots$

Grammar for $L_2 = \{a^n b^m \mid n, m \geq 0\}$

Start symbol: S

Terminals: {a, b}

Non-terminals: {S, A, B}

Production rule

$S \rightarrow AB$
$A \rightarrow aA \mid \varepsilon$
$B \rightarrow bB \mid \varepsilon$

Derivation for aabb

$S \Rightarrow AB$
$\Rightarrow aAB$
$\Rightarrow aaAB$
$\Rightarrow aaB$
$\Rightarrow aabB$
$\Rightarrow aabb$

Generated string $\varepsilon, a, b, ab, aabb \ldots$

Grammar for Arithmetic Expression

Production rule

$$E \rightarrow E+E \mid id$$

Start symbol : E

Terminals : {id, +}

Non terminals {E}

Derivation for id+id

$E = E+E$

$\Rightarrow id+E$

$\Rightarrow id+id$

Grammar for balanced parentheses

Production rule : $S \rightarrow (S)S \mid \varepsilon$

Derivation for (())

$S \Rightarrow (S)S$

$\Rightarrow ((S)S)S$

$= (())S$

$= (())$

Grammar for binary strings with equal zeros and ones

Production rule : $S \rightarrow 0S1S \mid 1S0S \mid \varepsilon$

Start symbol : S

Terminals : {0,1}

Non-terminals : {S}

Derivation for 01

$S \Rightarrow 0S1S$

$\Rightarrow 01S$

$\Rightarrow 01$

Task 4 - Derivation and

Given Grammar

$S \rightarrow aSb \mid \varepsilon$

Grammar Components

Start symbol : S

Terminals : {a, b}

Non-terminals : {S}

Production rule : $S \rightarrow aSb \mid \varepsilon$

Generated Strings

n	Generated string	Derivation
0	ε	$S \rightarrow \varepsilon$
1	ab	$S \rightarrow aSb \rightarrow ab$
2	aabb	$S \rightarrow aSb \rightarrow aaSbb$
3	aaabbb	$S \rightarrow aSb \rightarrow aaSbb$ $\rightarrow aaabbb$

Leftmost Derivation of aaabbb

In leftmost derivation the leftmost non-terminal is replaced first

$S \Rightarrow aSb$

$\Rightarrow aaSbb$

$\Rightarrow aaaSbbb$

$\Rightarrow aaabbb$

Rightmost derivation of aaabbb

$S \Rightarrow aSb$

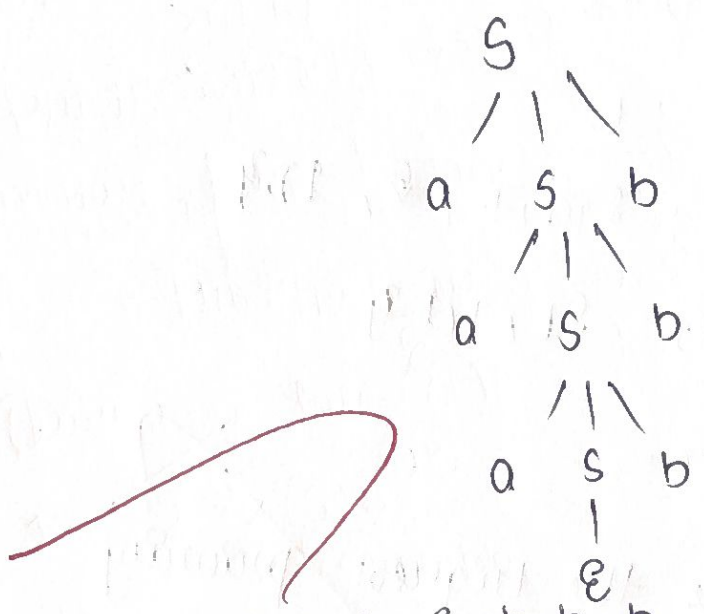
$\Rightarrow aaSbb$

$\Rightarrow aaaSbbb$

$\Rightarrow aaabbb$

Both derivations are identical because there is only one non-terminal at each step.

Parse tree for aaabbb



Leaf node : a, a, a, ε, b, b, b

APPLICATIONS

Understanding grammar structure

Syntax analysis in compilers

Parsing programming language statements

Checking valid input sequence.

Task 5- Grammar design for Arithmetic Expression

Context Free Grammar

Production rules

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid id$

Grammar Components

Start symbol : E

Terminals : {id, +, *, (,)}

Non-terminals : {E, T, F}

Operator precedence:

Paranthoses have the highest priority

Multiplication * has higher precedence than +

Addition has the lower precedence.

Derivation of a+b*c

Given expression

a+b*c

Derivation

$E \Rightarrow E+T$

$\Rightarrow T+T$

$\Rightarrow F+T$

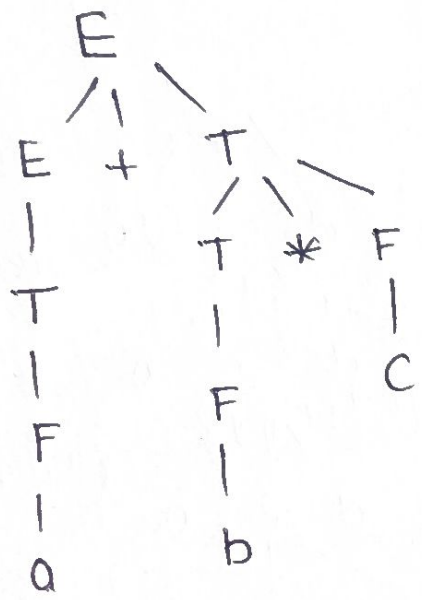
$\Rightarrow a+T$

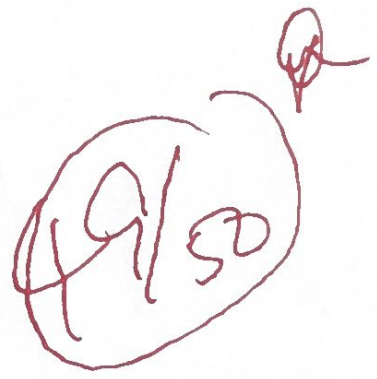
$\Rightarrow a+TF$

$\Rightarrow a+FF$

$\Rightarrow a+b*c$

Parse tree for a+b*c





Example expression

Expression	Validity
a+b	Valid
a*b	Valid
a+b*c	Valid
(a+b)*c	Valid
a*(b+c	Valid

APPLICATIONS

Compiler design

Arithmetic expression evaluation

Syntax checking.